Data Protection & Privacy in Cloud

Encryption • Secure Storage • GDPR / HIPAA • Anonymization

aws

Azure

Google Cloud

Why data protection matters (and what you will build)

Real-world goal

Build a secure “file upload + storage” pipeline that:

- Encrypts data at rest and in transit
- Uses managed keys (KMS/Key Vault/Cloud KMS)
- Prevents public access misconfigurations
- Logs access (audit trail)
- Supports privacy (minimize + anonymize where needed)

Typical mistakes (seen in internships)

1) Bucket/container made public “for testing”
2) Encryption turned off or unknown
3) Secrets hardcoded in code / .env
4) No rotation of keys
5) No logs → cannot investigate incidents

Mini-lab roadmap (90 minutes)

A) Create a storage bucket/container (private)
B) Turn on default encryption with a managed key (CMK/CMEK)
C) Upload using a time-limited URL/token (pre-signed URL / SAS)
D) Verify: public access prevention + audit logs + key usage logs

Encryption basics (practical view)

Data in transit (network)

- Use TLS (HTTPS) between client <-> server <-> storage
- Protects against sniffing / MITM
- Verify certificates + disable weak ciphers

Data at rest (stored)

- Encrypt files/objects on disk
- Cloud storage supports server-side encryption (SSE)
- Option: customer-managed keys for control & audit

Envelope encryption (how KMS works)

Plaintext (File / Record) → Data Key (AES) → Encrypted Data (ciphertext) → Encrypted Data Key (wrapped by KMS key)

Idea: encrypt data locally with a data key; then protect that data key by encrypting it with a long-term KMS key.

Key management services (KMS/Key Vault/Cloud KMS)

What they do (in one line): create keys, control who can use them, rotate/disable them, and log key usage.

AWS KMS

Best for: encrypting AWS data services

Practical features:
- KMS keys + data keys
- IAM policies control usage
- Audit via CloudTrail
- Integrates with S3, RDS, EBS...

Azure Key Vault

Stores: secrets, keys, certificates

Practical features:
- RBAC / access policies
- HSM-backed options
- Managed identities avoid hardcoded secrets
- Logging to Azure Monitor

Google Cloud KMS

Best for: CMEK (customer-managed keys)

Practical features:
- Key rings + keys
- IAM controls key usage
- Audit logs in Cloud Logging
- Integrates with Cloud Storage, BigQuery...

Key management services (KMS/Key Vault/Cloud KMS)

What they do (in one line): create keys, control who can use them, rotate/disable them, and log key usage.

Dimension	AWS KMS	Azure Key Vault	Google Cloud KMS
Main focus	**Keys** for encryption/signing	**Keys + Secrets + Certificates**	**Keys** for encryption/signing
How you organize keys	KMS keys (regional); policies/grants	Vault (or Managed HSM) contains objects	KeyRing → CryptoKey → versions (Google Cloud Documentation)
Access control model	Key policies + IAM + grants; audited in CloudTrail (AWS Documentation)	Azure RBAC / access model; vault-level controls (Microsoft Learn)	IAM roles per key/keyring/project (Google Cloud Documentation)
Rotation	Customer-managed symmetric keys: automatic rotation (configurable); AWS-managed rotate yearly (AWS Documentation)	Rotation supported; plus strong recovery controls (soft delete/purge protection) (Microsoft Learn)	Rotation schedules; key versions lifecycle (Google Cloud Documentation)
Logging/audit	CloudTrail logs for KMS API calls (AWS Documentation)	Diagnostic logs to Azure Monitor/Log Analytics (Microsoft Learn)	Cloud Audit Logs for KMS activities (Google Cloud Documentation)
"Secrets" storage	Use **Secrets Manager** (separate service)	**Built in** (secrets are first-class objects) (Microsoft Learn)	Use **Secret Manager** (separate service)
Best fit in real projects	Encrypt data in AWS services (S3/EBS/RDS etc.) with customer-managed keys; strict audit trail	Central "vault" for app secrets + keys + certs; great for app configuration/security	CMEK-style key control across GCP services; strong org/IAM integration

Mini-demo: envelope encryption in practice

Scenario

You want to encrypt a 50 MB file before storing it in cloud object storage.
Do NOT send the whole file to KMS. Instead:

Steps (cloud-agnostic)

1) Request a data key from KMS (returns plaintext + encrypted copy)
2) Encrypt file locally with plaintext data key (AES)
3) Store: (encrypted file) + (encrypted data key)
4) To decrypt: send encrypted data key to KMS → get plaintext key → decrypt file

Pseudo-code

```
dk = KMS.GenerateDataKey(keyId)
ciphertext = AES_Encrypt(fileBytes, dk.plaintext)
store(ciphertext, dk.encrypted)

dk2 = KMS.Decrypt(dk.encrypted)
fileBytes = AES_Decrypt(ciphertext, dk2.plaintext)
```

AWS practical: S3 + KMS (SSE-KMS)

What to configure

- Bucket: keep private + enable “Block Public Access”
- Default encryption: SSE-KMS
- IAM: allow app role to “PutObject/GetObject”
- KMS key policy: allow usage (encrypt/decrypt) only for app role

AWS CLI (concept)

```
# Create a customer-managed KMS key (CMK)
aws kms create-key

# Enable default SSE-KMS on a bucket
aws s3api put-bucket-encryption --bucket <B> --server-side-encryption-configuration ...

# Upload (S3 encrypts using KMS)
aws s3 cp file.pdf s3://<B>/file.pdf
```

Verification checklist (students must show in lab report)

- ✅ Bucket is not public (Block Public Access enabled)
- ✅ Default encryption is SSE-KMS (not “None”)
- ✅ KMS key has rotation policy (if required)
- ✅ Audit trail: CloudTrail shows who used the key / accessed bucket

Azure practical: Blob Storage + Key Vault

What to configure

- Storage account: disable anonymous blob access
- Encryption at rest: Storage Service Encryption (SSE)
- For extra control: customer-managed key in Key Vault
- Access: Azure AD RBAC + Managed Identity (avoid secrets)

Azure CLI (concept)

```
# Create a Key Vault + key
az keyvault create -n <KV> -g <RG>
az keyvault key create --vault-name <KV> -n cmk --protection software

# Storage encryption (SSE) is on by default; configure CMK if required
# Prefer Managed Identity for apps accessing storage
```

Verification checklist

- ✅ Anonymous blob access is disabled at account level
- ✅ Encryption at rest (SSE) enabled
- ✅ App uses Managed Identity + RBAC (no hardcoded secrets)
- ✅ Key Vault logging enabled (audit key access)

Google Cloud practical: Cloud Storage + Cloud KMS (CMEK)

Google Cloud

What to configure

- Bucket: enforce Public Access Prevention
- Default encryption: Google-managed by default
- For extra control: CMEK using Cloud KMS
- IAM: grant key usage only to service account used by your app

gcloud (concept)

```
# Create key ring + key
gcloud kms keyrings create <KR> --location=<LOC>
gcloud kms keys create cmek --keyring=<KR> --location=<LOC> --purpose=encryption

# Set bucket default CMEK
gcloud storage buckets update gs://<B> --default-encryption-
key=projects/.../cryptoKeys/cmek
```

Verification checklist

- ✅ Public Access Prevention is enforced
- ✅ Bucket uses default encryption key (CMEK if required)
- ✅ Audit logs show KMS key usage
- ✅ Service account permissions are minimal (least privilege)

Secure data storage (S3 / Blob / Cloud Storage): checklist

1) Prevent accidental public access

AWS: Block Public Access • Azure: disable anonymous blob access • GCP: Public Access Prevention

Access control

- Least privilege roles
- No shared accounts
- Short-lived access (SAS / pre-signed URLs)
- Separate dev/test/prod buckets

Encryption + keys

- Default encryption ON
- Use CMK/CMEK when needed
- Rotate keys (policy-driven)
- Separate duties (use vs admin)

Durability + recovery

- Versioning
- Immutability / WORM (Object Lock)
- Lifecycle policies
- Backup critical data

Privacy regulations (engineer’s view): GDPR & HIPAA

GDPR (EU) — key ideas

Applies when processing personal data of people in the EU.

Engineering takeaways:

- Data minimization (collect only what you need)
- Purpose limitation (use only for stated purpose)
- Security of processing (encryption + access control)
- Rights: access, erase, rectify (need data inventory)

HIPAA (US) — key ideas

Applies to protected health information (PHI) handled by regulated entities.

Engineering takeaways:

- Safeguards: administrative, physical, technical
- Focus: confidentiality, integrity, availability
- Access controls + audit controls
- Encrypt PHI in transit + at rest

Privacy-by-design (practical techniques)

Design moves that reduce privacy risk

In your API & database

- Collect minimum fields (no “nice to have” PII)
- Separate identifiers from sensitive attributes
- Encrypt sensitive columns/records
- Access logs: who read which record
- Retention policy: auto-delete old data

In cloud storage

- Store PII in a dedicated bucket/container
- Strict IAM + no public access
- Use time-limited uploads/downloads
- Tokenize filenames/paths (avoid names in URLs)
- Enable versioning + immutability if needed

Anonymization & privacy preservation (with examples)

Pseudonymization vs anonymization

Pseudonymization: replace identifiers but keep a mapping separately (reversible with extra info).
Anonymization: transform data so individuals are not identifiable (harder; risk-based).

Common techniques (engineering)

- Masking (hide part of value)
- Tokenization (replace with random token)
- Hashing + salt (for matching, not secrecy)
- Generalization (age → age band)
- k-anonymity / differential privacy (advanced)

Mini example: anonymize a student dataset for analytics

Field	Before	After (privacy-preserving)
Name	"Riya Sharma"	Token: U-8F2A9C
Phone	9876543210	Mask: 98******10
DOB	2004-03-21	Age band: 18–21
College	ABC Institute	Generalize: Tier-2

Case study + quick quiz

Case: “Public bucket leak” (common pattern)

Symptom:
- Someone shares a storage URL; files become accessible publicly

Root causes:
- Public access enabled (bucket policy / ACL / anonymous access)
- No guardrails (public access prevention)

Fix (order matters):
1) Block public access immediately
2) Rotate exposed credentials/tokens
3) Review logs (who accessed what)
4) Add preventive policies (deny public)
5) Add default encryption + key auditing

Quick quiz (5 mins)

1) Why is CMK/CMEK useful beyond encryption?
2) Give one safe alternative to “public bucket for sharing”.
3) What logs help you prove who accessed a file?
4) Pseudonymization vs anonymization (1 line each).
5) Name one GDPR principle that changes DB design.

Expected answers (brief):
kill switch + audit; pre-signed URL/SAS; audit/flow logs; reversible vs not; minimization/purpose limitation.